

GMOCAT 精读笔记：图增强的多目标计算机自适应测试

原文:3580305.3599367.pdf(KDD '23,Hangyu Wang 等,上海交大/吉林大学/华为)·代码:<https://github.com/justarter/GMOCAT>
(已克隆到 GMOCAT-code/; 官方 MindSpore 仓库只有占位 README, PyTorch 版才是真正可跑的实现)·笔记于
2026-06-26

一、解决什么问题

先把 CAT (计算机自适应测试) 的大背景捋清楚。CAT 就是“边考边出题”的在线考试系统, 它有两个轮流工作的核心部件:

- **认知诊断模型 (CDM)**: 根据学生已经答过的题, 估计他当前的能力 θ 。最经典的是 IRT, 深一点的是 NeuralCDM。
- **选择算法 (Selection Algorithm)**: 根据历史作答, 决定下一题考哪道。

本文只动“选择算法”这一半 (CDM 直接用现成的 IRT / NCD)。

痛点在哪? 以前的数据驱动选题算法 (BOBCAT 用元学习、NCAT 用强化学习) 都只盯着一个目标——**质量 (quality)**, 即“用最少的题把学生能力估得最准”。作者指出这在现实中不够, 还差两件事:

1. **多样性 (diversity)** 不足: 好的考试应覆盖多个知识点 (期末数学卷子要同时考代数、几何)。只追准确率的算法往往反复在少数几个概念上出题。
2. **新颖性 (novelty)** 不足: 题库就那么小, 热门题被反复推给不同学生, 导致**题目过度曝光**——题目泄露、后面的学生提前知道答案、分数虚高。

而且, 以前的方法都没用上**题目与知识概念之间的关系信息** (哪道题考哪些概念、概念之间谁是谁的先修)。

GMOCAT 的两个核心改进: - 把 **质量 / 多样性 / 新颖性** 三个目标统一进一个**标量化多目标强化学习 (Scalarized MORL)** 框架, 各自配一个奖励。这是第一个把 MORL 用进 CAT 的工作。- 用**图注意力网络 (GAT)** 在“**题目—概念相关图**”和“**概念—概念先修图**”上学习**关系感知**嵌入, 让选题更聪明。也是第一个把关系图引入 CAT 的工作。

二、问题定义 (输入 / 输出)

把 CAT 选题建模成一个多目标马尔可夫决策过程 (**MOMDP**) $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ 。注意这里 RL 的“智能体”就是**选择算法**, “环境”就是学生 + CDM。

- **状态 s_t** : 把学生前 $t - 1$ 步的作答记录 $\{(q_1, c_1, y_1), \dots, (q_{t-1}, c_{t-1}, y_{t-1})\}$ 编码成的一个向量 (q = 题目, c = 概念, y = 对/错)。由状态编码器 f_{se} 产出, 维度 $D = 3d_s$ 。
- **动作 q_t** : 从候选题集 $\mathcal{D}_c$ 里选出的下一道题。动作空间 = 题库。
- **奖励 $r(s_t, q_t) = [r_{qua}, r_{div}, r_{nov}]$** : 关键——**奖励是一个三维向量**, 不是一个标量。分别衡量“这一步选题对准确率/多样性/新颖性的贡献”。
- γ : 折扣因子, 论文取 0.5。

目标: 学一个策略 $\pi(q_t | s_t)$, 最大化加权回报 (公式 1/2)。

一个具体小例子 (贴 KT/CAT 场景): 假设题库里 5 道题 $q_1 \dots q_5$, 最大测试长度 $T = 20$ (论文设定)。学生小明真实能力 θ^* 未知且固定。
- $t = 1$: 历史为空, 算法选了 q_3 (考“分数加法”), 小明答对, $y = 1$ 。CDM 把能力估计从初始更新成 θ_1 。
- $t = 2$: 状态 s_2 编码了 (q_3 , 分数加法, 1), 算法选 q_1 (考“几何”), 小明答错。- ...如此 20 步。每一步, 环境返回一个三维奖励。

最终的“输出”有两层: - 训练时: 输出是学到的选题策略 π 。- 测试时: 对新学生, 策略每步选一道题; 20 步后用 **AUC/ACC** (在元题集 $\mathcal{D}_m$ 上预测对错) 衡量能力估得准不准, 用 **Cov** 衡量概念覆盖, 用 **曝光率/重叠率** 衡量新颖性。

注意一个数据划分细节: 每个学生的题被分成**候选集 $\mathcal{D}_c$ (80%, 能被选)**和**元题集 $\mathcal{D}_m$ (20%, 只用来算质量奖励、不会被考)**。

三、模型结构

GMOCAT 四大模块, 数据流是: **关系聚合器** $\rightarrow$ **状态编码器** $\rightarrow$ **Actor-Critic 推荐器**, 外加一个多目标奖励给整个 RL 当老师。对照代码 function/GCAT.py 的 ActorCritic 类看非常清楚。

第 1 步: 关系聚合器 (Relation Aggregator, 对应 4.5 节 / 代码 Fusion + GraphLayer) - 输入: 题目的原始嵌入 E_q 、概念的原始嵌入 E_c (就是两张 nn.Embedding 查表)。- 处理: 在两张图上跑 GAT。- 概念在**先修图**和**相关图**里都出现, 所以概念嵌入吸收两种邻居信息, 再用一个注意力权重把两路加权融合。- 题目只在相关图里, 所以题目嵌入只吸收它相关概念的信息。- 输出: 关系感知嵌入 $\tilde{E}_q, \tilde{E}_c$ 。代码里就是 `kn_emb2, exer_emb2 = self.graphencoder(kn_emb, exer_emb)`。

第2步：状态编码器(State Encoder, 对应4.3节 / 代码 hidden_layer + Encoder) - 输入：前 $t-1$ 步的记录, 每步用 $\tilde{E}_q^{t'} \oplus \tilde{E}_c^{t'} \oplus E_y^{t'}$ 拼成一个 $3d$ 维向量 (代码 `torch.cat([batch_exer_emb, batch_ans_emb, batch_kn_emb], dim=-1)`)。- 处理：堆成矩阵 E_t , 过自注意力 (Transformer encoder, 让“答对难题”这种高信息量记录获得更大权重), 再平均池化。- 输出：状态向量 $s_t \in \mathbb{R}^D$ 。代码 `hidden_state = torch.sum(atten_hidden, dim=1) / ...`。- 一个巧妙点：因为学生真实能力在考试中不变, 所以记录的先后顺序不重要 $\rightarrow$ 用平均池化而非 RNN, 也不需要位置编码。

第3步：Actor-Critic 推荐器 (对应4.4节) - Actor (actor_layer)：吃状态 s_t , 输出题库上的一个概率分布 $\pi(q_t|s_t)$, 采样出下一题。- **Critic (acc_head / div_head / nov_head 三个头)**：吃状态 s_t , 输出一个三维价值向量 $V(s_t) = [V_{qua}, V_{div}, V_{nov}]$ ——每个目标一个价值估计, 这就是“向量化 critic”。- 用多目标版 PPO (MOPPO) 训练。

多目标奖励 (4.2节) 贯穿始终, 给 RL 当监督信号, 下面详解。

四、关键公式详解

公式 1/2：多目标优化目标

$$\max_{\pi} J = \max_{\pi} \mathbb{E}_{i \sim \pi} \left[w^T \left(\sum_{t'=1}^T \gamma^{t'} r(s_{t'}^i, q_{t'}^i) \right) \right]$$

- 符号： $r(\cdot) \in \mathbb{R}^3$ 是三维奖励向量； $w \in \mathbb{R}^3$ 是标量化函数/权重向量, 每一维代表一个目标的重要程度； γ 折扣。 $w^T(\cdot)$ 把三维回报压成一个标量来最大化。- 举例：设 $w = [1, 1, 1]$ (论文默认), 某条 20 步轨迹三个目标的折扣累计回报分别是 $[2.5, 6, 14]$ (质量涨了 2.5 的累计 ACC、覆盖了 6 个新概念、14 步选的是冷难题)。则 $w^T \cdot [2.5, 6, 14] = 22.5$ 。若改成 $w = [1, 0, 2]$ (更在意新颖性、不要多样性), 则 $= 2.5 + 0 + 28 = 30.5$, 策略会更倾向选冷难题。- 作用：一个统一的旋钮 w , 让实践者按需在三个目标间权衡。

公式 3：质量奖励

$$r_{qua} = ACC(\theta_t) - ACC(\theta_{t-1})$$

- 符号： $ACC(\theta_t)$ 是用第 t 步的能力估计 θ_t 在元题集 $\mathcal{D}_m$ 上预测对错的准确率。- 举例：元题集有 10 道题。选 q_t 并让学生作答、CDM 更新后, θ_t 在这 10 道上预测对 7 道 $\rightarrow ACC(\theta_t) = 0.7$ ；上一步只对 6 道 $\rightarrow ACC(\theta_{t-1}) = 0.6$ 。则 $r_{qua} = 0.7 - 0.6 = 0.1$ (正向激励, 说明这题选得好, 让能力估计更准了)。若反而掉到 0.5, $r_{qua} = -0.1$ (惩罚)。- 作用：直接奖励“让能力估计更接近真实”的选题。是个差分/增量奖励, 所以累计起来就是总的准确率提升。- 代码： `envs/GCATEnv.py` 第 104 行 `acc_rwd = new_accuracy - self.last_accuracy`, 一一对应。

公式 4：多样性奖励

$$r_{div} = \begin{cases} 1, & c_t \setminus \{c_1 \cup c_2 \cdots \cup c_{t-1}\} \neq \emptyset \\ 0, & \text{otherwise} \end{cases}$$

- 符号： c_t 是第 t 题考的概念集合；花括号里是前面所有题已经考过的概念并集。差集非空 = 这道题带来了至少一个新概念。- 举例：前几步已考过概念 { 加法, 几何 }。这一步选的题考“分数加法 + 概率”, 其中“概率”是新的 $\rightarrow$ 差集 = { 概率 } $\neq \emptyset \rightarrow r_{div} = 1$ 。若这题只考“加法、几何”, 全是旧的 $\rightarrow r_{div} = 0$ 。- 作用：鼓励不断引入新知识点, 提高概念覆盖率 Cov。二值奖励简单稳定。- 代码： `compute_div_reward()` (GCATEnv 第 127 行), 遍历该题概念, 只要有一个不在 `tested_questions` 覆盖的概念里就 `reward=1`。

公式 5：新颖性奖励

$$r_{nov} = \begin{cases} 1, & q_t \notin \mathcal{T} \\ 0, & \text{otherwise} \end{cases}$$

- 符号： $\mathcal{T}$ = 训练集中最热门的前 $x\%$ 题目集合 (论文 $x = 10$)。 $\mathcal{T}$ 在训练前算好就固定不变。- 举例：统计训练集发现 q_7 是被选最多的 top-10% 之一 $\rightarrow q_7 \in \mathcal{T} \rightarrow$ 若这步选 q_7 , $r_{nov} = 0$ (不鼓励)。若选了冷门的 $q_{42} \notin \mathcal{T}$, $r_{nov} = 1$ 。- 作用：直接把“少选热门题”写进奖励, 从源头压低题目曝光率, 避免泄题。这是 GMOCAT 比 NCAT (靠玻尔兹曼采样间接控曝光) 更直接的地方。- 代码： `nov_reward_map` (从 `data/nov_reward*.json` 读入), `nov_rwd = self.nov_reward_map[action]`。- 三个奖励在代码里拼成 (B,3) 的向量： `rwd = np.concatenate((acc_rwd, div_rwd, nov_rwd), axis=-1)` ——这就是论文说的“向量值奖励”。

公式 6 / 7: 状态编码

$$e_{t'} = \tilde{E}_q^{t'} \oplus \tilde{E}_c^{t'} \oplus E_y^{t'}, \quad \tilde{E}_t = \text{Softmax}\left(\frac{(E_t W^Q)(E_t W^K)^T}{\sqrt{d_k}}\right)(E_t W^V)$$

- 符号: $\oplus$ 拼接, $D = 3d$ (论文 $d = 128$, 故 $D = 384$)。公式 7 就是标准 Transformer 自注意力, $W^Q, W^K, W^V \in \mathbb{R}^{D \times D}$ 。- 举例: 第 3 步时历史有 2 条记录 $\rightarrow E_t$ 是 2×384 矩阵。自注意力让“答对难题”那条记录拿到更高注意力权重, 输出还是 2×384 , 再平均池化成 384 维的 s_3 。- 作用: 把变长历史压成长状态, 且能区分不同记录的信息量。- 代码: `MultiHeadedAttention / attention()` (GCAT.py 372 行), 与公式 7 的缩放点积一字不差。

公式 8-11: 多目标 PPO (MOPPO)

$$A(s_t, q_t) = \sum_{t'=t} \gamma^{t'-t} r(s_{t'}, q_{t'}) - V(s_t) \quad (\text{向量}, 8)$$

$$L_1 = -\mathbb{E} \left[\min \{ \rho w^T A, \text{Clip}(\rho, 1 - \epsilon, 1 + \epsilon) w^T A \} \right], \quad \rho = \frac{\pi(q_t | s_t)}{\pi_{old}(q_t | s_t)} \quad (9)$$

$$L_2 = \frac{1}{2} w^T \|V(s_t) - \sum_{t' \geq t} \gamma^{t'-t} r(s_{t'}, q_{t'})\|^2 \quad (10), \quad L = L_1 + \alpha L_2 \quad (11)$$

- 符号: A 是三维优势向量 (实际回报 - critic 预测); ρ 是新旧策略比; $\epsilon = 0.2$ 裁剪; $w^T A$ 把三维优势压成标量再喂给 PPO; $\alpha = 1.0$ 平衡 actor/critic 损失。- 举例: 某步三维实际折扣回报 = $[0.2, 1, 1]$, critic 预测 $V = [0.1, 0.8, 0.7] \rightarrow A = [0.1, 0.2, 0.3]$ 。用 $w = [1, 1, 1] \rightarrow w^T A = 0.6 > 0$, 说明这步比预期好, PPO 会提高选这道题的概率 (但 ρ 被裁剪在 $[0.8, 1.2]$ 内防止步子太大)。critic 损失 L_2 则把 V 往 $[0.2, 1, 1]$ 拉近。- 作用: 先向量化、最后才标量化是本文的关键设计——critic 给每个目标单独估值, 不会把三个目标的信号混在一起, 比“一上来就把奖励加成一个标量” (即消融里的 GMOCAT-S) 效果好。- 代码: `GCAT.update()` (GCAT.py 166 行): `advantages = b_rewards - state_values` (公式 8, $B \times 3$), 再 `advantages[:, 0] * w0 + advantages[:, 1] * w1 + advantages[:, 2] * w2 (w^T A)`; `surr1/surr2 + torch.clamp` 就是公式 9; `critic_loss = square(state_values - b_rewards) * 0.5` 再按 w 加权就是公式 10。代码还多加了 $-0.01 * \text{entropy}$ 的熵正则鼓励探索 (论文正文没强调)。

公式 12-17: 图注意力聚合

$$g_{pre} = \sum_{c' \in N_c^{pre}} \alpha_{c,c'} W_{pre} E_{c'}, \quad \alpha_{c,c'} = \text{Softmax}(\text{att}_{pre}([W_{pre} E_c, W_{pre} E_{c'}]))$$

$$\mu_{pre} = P^T \tanh(W g_{pre} + b), \quad \tilde{E}_c = \mu_{pre} g_{pre} + \mu_{cor} g_{cor}$$

- 符号: N_c^{pre} 是概念 c 在先修图里的邻居; α 是 GAT 注意力权重 (邻居与 c 越相似权重越大); g_{pre}/g_{cor} 是从先修图/相关图聚来的两路表示; μ_{pre}, μ_{cor} 是融合这两路的注意力权重 (softmax 归一)。- 举例: 概念“分数加法”的先修邻居是“整数加法、分数概念”, 相关邻居是几道分数题。GAT 发现“分数概念”与它更相似 $\rightarrow$ 给更高的 α 。最后若模型判断先修信息更重要 $\rightarrow \mu_{pre} = 0.7, \mu_{cor} = 0.3 \rightarrow \tilde{E}_c = 0.7 g_{pre} + 0.3 g_{cor}$ 。题目嵌入更简单: 只从相关图聚合 (公式 17), $\tilde{E}_c = h_{cor}$ 。- 作用: 让“分数加法”的表示天然带上“它依赖整数加法”这类结构知识, 使选题能利用概念间的拓扑关系 (比如先考先修概念)。- 代码: `GraphLayer` (GCAT.py 230 行) 就是一层 GAT——`edge_attention` 算 α (leaky_relu 对应论文的 LeakyReLU), `reduce_func` 里 softmax + 加权求和。`Fusion.forward` (284 行): `k_directed` = 先修图聚合 (A), `k_from_e_graph` = 相关图聚合 (B), `relation_attn + softmax` 算 μ 把 A、B 融合 (公式 15-16); `exer_emb = e_from_k_graph` 对应公式 17。

五、代码实现对照 (仓库 justarter/GMOCAT, PyTorch + dgl)

论文概念/公式	代码位置
Actor-Critic 主模型	<code>function/GCAT.py</code> $\rightarrow$ <code>class ActorCritic</code>
三个 critic 头 (向量价值, 公式 4.4)	<code>acc_head / div_head / nov_head, evaluate()</code> 里拼成 <code>morl_v</code> ($B \times 3$)
状态编码: 拼接 (公式 6) + 自注意力 (公式 7) + 平均池化	<code>ActorCritic.hidden_layer()</code>
关系聚合 GAT (公式 12-14)	<code>class GraphLayer (edge_attention/reduce_func)</code>
先修/相关两路融合 (公式 15-16)	<code>class Fusion.forward + relation_attn</code>
题目只走相关图 (公式 17)	<code>Fusion: exer_emb = e_from_k_graph[0:item_num]</code>
MOPPO 优势 + 裁剪损失 (公式 8-11)	<code>GCAT.update()</code> (标量化 <code>advantages * morl_weights</code>)
质量/多样性/新颖性奖励 (公式 3-5)	<code>envs/GCATEnv.py: acc_rwd / compute_div_reward / nov_reward_map</code>

论文概念/公式	代码位置
先修图构建 (附录 B)	<code>construct_graphs.py</code>
CDM (IRT/NCD 环境)	<code>envs/irt.py</code> 、 <code>envs/ncd.py</code>
入口/超参	<code>launch_gcat.py</code> 、 <code>train_gcat.sh</code> (仓库已附 <code>assist2009</code> 预处理数据, 可直接在 IRT 上跑)

代码与论文高度一致, 唯一值得注意的小差异: actor 损失里多了 $-0.01 * \text{entropy}$ 熵正则项 (鼓励探索), 论文正文未提。

六、小结

实验结论 (三数据集 Eedi / ASSIST / Junyi, 两种 CDM IRT/NCD): - **质量**: AUC/ACC 全面超过 Random、MFI、KLI、MAAT、BOBCAT、NCAT, 最高约 +2% AUC, 且带统计显著性。RL 类方法 (GMOCAT、NCAT) 在测试后期 (step 20) 稳压贪心的 MAAT, 印证“长期视角”的价值。- **多样性**: Cov 曲线涨得最快。- **新颖性**: 在 Junyi 上曝光率 (>0.2) 和重叠率都最低 (除了完全不个性化的 Random)。- **消融**: 去掉关系图 (-R) 或去掉向量化标量函数 (-S) 性能都下降 $\rightarrow$ 两个核心设计都有效。- **目标权衡** (5.7 节): 三目标之间既促进又冲突。有意思的发现——加多样性目标**反而提升了** AUC/ACC (因为能力是多面的, 多覆盖概念估得更准); 而加新颖性目标会**轻微拉低**质量 (被迫选了些低质量冷门题)。w 给了实践者按需调节的旋钮。

优点: 首个把多目标 RL + 关系图统一进 CAT; 奖励设计直观; “先向量化、最后标量化”比朴素加权更稳; 代码开源可复现。

局限: w 目前靠人手调 (0/1 网格), 作者把“自动、定量地平衡多目标”留作未来工作; 新颖性与质量存在固有冲突; 先修图对没有标注先修关系的数据集需用启发式 (附录 B) 构造, 质量依赖该构造。